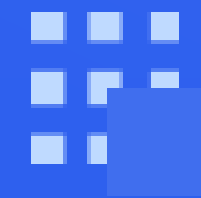


软件测试

均值方差程序MeanVar



MeanVar的需求规格

需求描述：输入一个长度为 L 的数组 X ，返回数组的平均值 $mean$ 和方差 var 。

隐含需求：（1）数组中的每个元素 $X[i]$ 都是实数；（2） $L \geq 1$

$$\text{均值计算公式: } mean = \frac{\sum_{i=1}^n x_i}{n}$$

方差的计算公式:

$$var = \frac{\sum_{i=1}^n (x_i - mean)^2}{n}$$

$$var = \frac{\sum_{i=1}^n (x_i - mean)^2}{n - 1}$$

方差与修正方差的区别

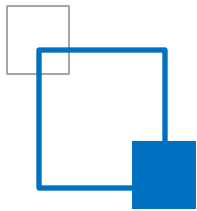
MeanVar的C程序

```
#include <stdio.h>
void MeanVar(double X[], int L) {
    double var, mean, sum, varsum;
    sum = 0;
    for (int i = 0; i < L; i++) {
        sum += X[i];
    }
    mean = sum / L;
    varsum = 0;
    for (int i = 0; i < L; i++) {
        varsum += ((X[i] - mean) * (X[i] - mean));
    }
    var = varsum / (L - 1);
    printf("Mean: %f\n", mean);
    printf("Variance: %f\n", var);
}
```

程序循环结构与对应的测试策略

MeanVar的Java程序

```
public class MeanVar {
    public static void MeanVar(double[] X) {
        double var, mean, sum, varsum;
        sum = 0;
        int L = X.length;
        for (int i = 0; i < L; i++) {
            sum += X[i];
        }
        mean = sum / L;
        varsum = 0;
        for (int i = 0; i < L; i++) {
            varsum += ((X[i] - mean) * (X[i] - mean));
        }
        var = varsum / (L - 1);
        System.out.printf("Mean: %f\n", mean);
        System.out.printf("Variance: %f\n", var);
    }
}
```



MeanVar的测试用例示例

用例编号	测试输入	预期输出	实际输出
1	[1, 2, 3, 4, 5], 5	Mean=3.0, Var=2.5	
2	[1.5, 2.5, 3.5, 4.5], 4	Mean=3.0, Var=1.25	
3	[1, 1, 1, 1], 4	Mean=1.0, Var=0.0	
4	[0, 0, 0, 0], 4	Mean=0.0, Var=0.0	
5	[], 0	NAN	
6	[1], 1	Mean=1.0, Var=0.0	
7	[1, 2, 3, 4, -5], 5	Mean=1.0, Var=11.0	
8	[1.1, 1.2, 1.3, 1.4, 1.5], 5	Mean=1.3, Var=0.03	
9	[-1, -2, -3, -4, -5], 5	Mean=-3.0, Var=2.5	
10	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 10	Mean=5.5, Var=9.1666667	

等价类测试与边界值分析

$$var = \frac{\sum_{i=1}^n (x_i - mean)^2}{n}$$

基本公式先求出数据的平均数，再计算每个数据与平均数的差的平方，最后求这些平方差的平均值。公式直观，概念清晰，能准确反映方差的定义和本质，适用于数据量较小且对计算过程理解要求较高的情况。当数据量较大时，计算每个数据与平均数的差的平方以及求和等操作会耗费较多时间和精力，计算效率较低。

$$var = \frac{\sum_{i=1}^n x_i^2}{n} - mean^2$$

计算公式是计算二阶矩然后直接减去一阶矩（也就是平均值）的平方。可以证明两个公式等价。相比基本公式，减少了计算每个数据与平均数差值的步骤，在一定程度上提高了计算效率，尤其对于数据量较大的情况，能明显减少计算量。

方差计算的增量式方法

增量式算法是一种在不断有新数据加入时，能够高效更新方差的算法，避免了每次都重新计算整个数据集的方差，其核心思路是利用已有的方差来计算新数据加入后的方差。

$$\text{计算新的均值: } \bar{x}_{n+1} = \frac{n\bar{x}_n + x_{n+1}}{n+1}。$$

$$\text{计算新的方差: } s_{n+1}^2 = \frac{n}{n+1} s_n^2 + \frac{n}{(n+1)^2} (\bar{x}_n - x_{n+1})^2$$

在处理实时数据或数据不断增加的情况时，无需重新计算整个数据集的方差，只需利用已有的统计信息和新数据进行简单计算，大大减少了计算量，提高了计算效率。适用于各种数据类型和应用场景，无论是在线数据处理、实时监控，还是机器学习中的增量学习等领域，都能方便地应用该算法来及时更新方差，以反映数据的最新特征。

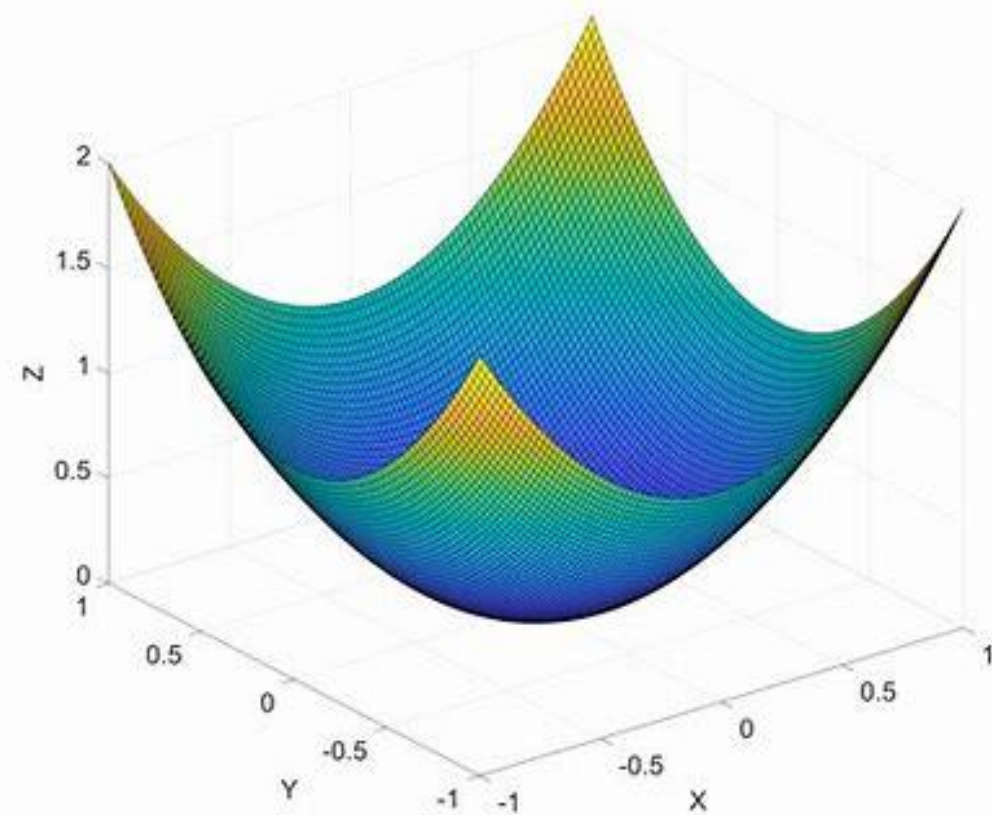
方差计算的增量式算法

```
double variance(double data[], int n) {  
    double mean = 0;  
    double M2 = 0;  
    for (int i = 0; i < n; i++) {  
        double x = data[i];  
        double delta = x - mean;  
        mean += delta / (i + 1);  
        M2 += delta * (x - mean);  
    }  
    double variance = M2 / (n - 1);  
    return variance;  
}
```

- 在进行大规模数值计算方差时，浮点运算可能会出现精度损失的问题。因为计算机使用二进制进行浮点数的存储和计算，在一定程度上会出现舍入误差。当进行大规模计算时，这些小的舍入误差会逐渐积累，导致计算结果与实际结果存在较大的误差。
- 为了避免精度损失的问题，可以选择使用高精度计算方法或者使用数值稳定的方差计算方法，例如增量式算法。也可以使用任意精度计算库来进行高精度计算，这些库可以提供更高的数值精度和计算准确性。
- 在进行大规模数值计算方差时，需要特别注意数值精度的问题，避免出现精度损失的问题。在选择计算方差的方法时，需要综合考虑计算速度和计算精度等因素，选择适合自己应用场景的计算方法。

课后实验：对比不同的方差计算程序，要求 $X[i]$ 数值很大，但 $X[i]$ 数值很接近，即偏差很小

回顾与讨论



测试设计与测试策略

- 循环结构对测试的影响
- 数值计算对测试的影响
- 不同算法对测试的影响